A Multi-Dimensional, Per-Pass Empirical Study of the LLVM Optimization Pipeline

Federico **Bruzzone**, Walter **Cazzola** • University of Milan, Italy • Computer Science Department

LLVM Developers' Meeting • Santa Clara, California • October 26–28, 2026

83% of per-pass impact comes from just **10 of 113** passes

84.8% of the pipeline is needed for 80% of the speedup (median)

29/30 kernels where stopping earlier is faster, at no larger binary

46% of the ideal speedup is lost to phase interference (mean)

Motivation

You pass `-O3` and your code gets faster. But *which* of the 113 passes actually did something? We decompose the `-O3` pipeline into cumulative per-pass prefixes and measure multiple dimensions on PolyBench/C kernels.

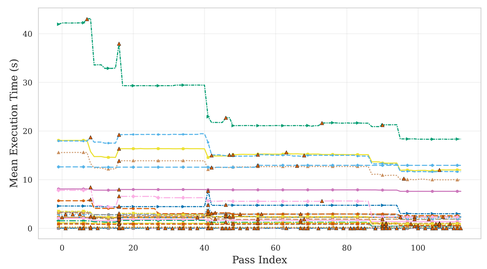
Study Design

- **Setup:** 113 passes of LLVM 21.1.8 `-O3` on 30 PolyBench/C 4.2.1 kernels
- **Method:** cumulative prefixes $P_i = p_0 \to \cdots \to p_i$; B_i vs. B_{i-1} gives the marginal impact of p_i
- **Metrics:** runtime, compile time, binary size, 9 HW counters (IPC, caches, branches, RAPL energy)
- **Rigor:** 84,750 runs on isolated cores; 95% CI < 1% of mean

`llvm-passview` — Open-Source Infrastructure
Every IR, binary, and profile, open on GitHub & Zenodo.

Non-Monotone & Back-Loaded Pipeline

`-O3` is not monotonically improving: 6.6–9.7% of pass-to-pass transitions *regress*. Three kernels (`correlation`, `covariance`, `seidel-2d`) end up *slower* than `-O0`.

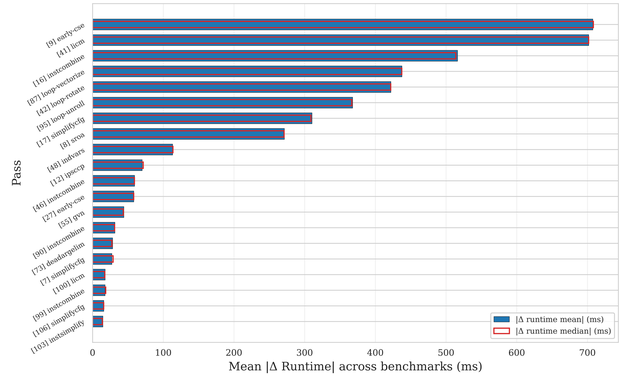


Back-loaded: the median kernel needs **84.8%** of the pipeline to lock in 80% of its final speedup, driven by `loop-vectorize` (pass 87) and `loop-unroll` (pass 95).

Phase interference: passes partly cancel each other's gains, losing **46.35%** of the ideal speedup (median 39.3%). On `correlation`, `licm` regresses runtime **26%** and `loop-rotate` recovers **25%** the very next pass.

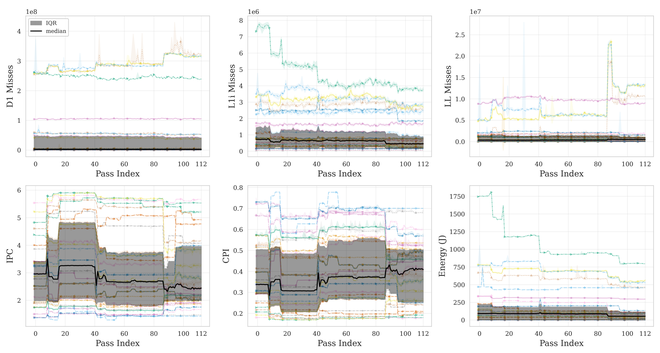
A Few Passes Do the Heavy Lifting

10 of 113 passes account for **83%** of per-pass runtime impact, led by `early-cse`, `licm`, `instcombine`, `loop-vectorize`, and `loop-rotate`. The bottom half contributes **less than 4%**.



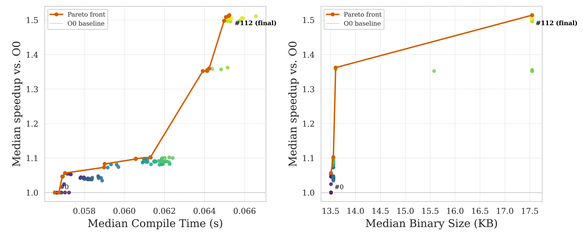
Cache Misses Stay Flat; IPC Does Not

Across all 113 passes (median & IQR over 30 kernels), **cache-miss medians barely move**, while **IPC steps down** at `licm` (pass 41) and `loop-vectorize`/`loop-unroll` (passes 87/95): overall **IPC falls 17.4%** even as runtime drops. **Energy tracks runtime** almost 1:1 (30–60% RAPL savings), the first per-pass energy profile of an LLVM pipeline.



Full `-O3` Is Not Pareto-Optimal

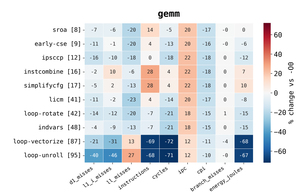
Most speedup arrives late in the pipeline for little extra compile time (left), while late inlining and unrolling inflate the binary (right). In **29 of 30** kernels, an earlier checkpoint is *faster* at *no larger* binary size.



Case Study: `gemm`

Matrix multiply: % change vs. `-O0`, 9 HW counters, top-10 passes.

```
for (i = 0; i < NI; i++) {
  for (j = 0; j < NJ; j++)
    C[i][j] *= beta;
  for (k = 0; k < NK; k++)
    for (j = 0; j < NJ; j++)
      C[i][j] += alpha * A[i][k] * B[k][j];
}
```



Takeaways

- ✔ **Pass pruning:** bottom-half passes give <4% of the impact
- ✔ **Autotuning:** per-pass deltas are dense supervision, no search
- ✘ **IR size ≠ runtime** (ρ=0.31): a poor cost-model proxy
- ✔ **Energy ≈ runtime:** optimize one, get both

Paper (arXiv)

`https://arxiv.org/abs/2606.31238`

Blog Post

federicobruzzone.github.io